

Highlights

Two-stage MILP scheduling and structure-aware learning-enhanced branch-and-cut for dual-source mixed-flow rotating-chamber cluster tools

Anonymous author(s)

- A two-stage MILP represents dual-source mixed-flow rotating-chamber operations.
- Conditional refinement preserves makespan and reshapes schedule timing.
- SPBS cuts mean PDI by 26.19% and raises incumbent availability to 100%.
- Role features and structure-conditioned beam decoding extend adapted HEM.
- RDMCT-HBS records the lowest observed mean PDI and time in 2700 nominal runs.

Two-stage MILP scheduling and structure-aware learning-enhanced branch-and-cut for dual-source mixed-flow rotating-chamber cluster tools

Anonymous author(s)

Abstract

Dual-source mixed-flow rotating-chamber cluster tools coordinate product wafers and reusable vacuum-zone dummy wafers across rotating chambers, load locks, and shared robots. We formulate a finite-horizon two-stage MILP that jointly determines $4 \times 1/2 \times 2$ recipe grouping, chamber and slot assignment, transfer, cleaning, and dummy-wafer allocation. Stage one minimizes makespan; stage two retains its makespan and discrete structure while refining timing. RDMCT-HBS accelerates branch-and-cut using a structure-preserving binary-skeleton (SPBS) warm start, a 23-dimensional variable-role representation, A3C-inspired parallel policy-gradient training, bounded beam decoding, and role-submodular completion. Four verification cases reach zero gap and pass independent checks. In a 2700-run nominal matrix, RDMCT-HBS attains the lowest mean primal–dual integral (1.6586×10^4) and solve time (218.32 s) among seven methods, reducing mean PDI by 1.41% relative to adapted HEM and 1.97% relative to SCIP. In a 960-run experiment, SPBS raises incumbent availability from 51.67% to 100% and reduces mean PDI by 26.19%; both effects remain significant after Holm adjustment. A 1500-run controlled study shows that the full two-stage profile reduces event-reconstructed mean route wait by 51.91%, maximum route wait by 68.89%, cadence CV by 39.70%, and cadence-deviation sum by 94.05% relative to disabling refinement; all four comparisons remain significant after Holm adjustment. Results establish advantages of the learning-enhanced search and two-stage scheduling.

Keywords: rotating-chamber cluster tool, semiconductor manufacturing, cluster-tool scheduling, mixed-integer programming, learned cut selection, parallel policy gradient, beam search

1. Introduction

Semiconductor manufacturers face high product variety, stringent process requirements, and strong pressure to obtain more capacity from installed equipment. Because capacity expansion is capital intensive, better operational use of existing tools remains an economically relevant alternative to adding hardware (Chien and Lan, 2021; Hong et al., 2026). Cluster tools are central to this objective because they integrate process modules, load locks, and robotic handlers within a tightly coupled environment. Their productivity is governed not only by nominal process durations but also by the coordination of material release, chamber occupancy, transport, cleaning, and temporal restrictions.

This study considers a dual-source mixed-flow rotating-chamber cluster tool deployed in a plasma-enhanced chemical vapor deposition (PECVD) process scenario. Product wafers arrive from load ports, whereas reusable vacuum-zone dummy wafers are supplied from a dedicated vacuum-zone dummy-wafer storage area. The two flows meet in shared rotating chambers and must be coordinated through atmospheric and vacuum robots and two load-lock paths. Mixed 4×1 and 2×2 recipes further couple product grouping, chamber and slot assignment, dummy-wafer availability, cleaning boundaries, and residency limits. A locally attractive transfer or processing order may consequently block several downstream resources. Operational complexity therefore arises from the propagation of decisions across the complete equipment, not from an isolated machine sequence.

Analytical and optimization-based cluster-tool studies have made substantial progress on steady-state cycles, multiple wafer types, chamber cleaning, and residency-time control (Kim et al., 2021; Ko et al., 2021; Qiao et al., 2022; Lee et al., 2023; Li et al., 2022; Wang et al., 2023a; Li and Yang, 2025; Yang et al., 2025). These models clarify important local mechanisms, but their system assumptions do not jointly represent the finite-horizon dual-source flow, mixed rotary recipes, dummy-wafer reuse, and shared transport architecture considered here. The multi-finger, multi-space model of Gu et al. (2024), for example, evaluates predetermined steady-state robot sequences, whereas the present problem must determine discrete assignments, resource orders, and continuous event times simultaneously.

Learning-based scheduling provides a complementary research stream. Deep reinforcement learning has been combined with genetic or adaptive search for dynamic semiconductor and cluster-tool decisions (Chien and Lan,

2021; Liu et al., 2022; Zhang et al., 2024; Kim and Lee, 2025; Kim and Kim, 2025). Particularly relevant is the simulation-integrated DDPG study
 40 of Hong et al. (2026), which adjusts continuous process times in a physical vapor deposition (PVD) cluster tool. That study demonstrates the value of linking equipment behavior with learned control, but its decision layer differs from the present one: the PVD policy regulates process duration under a simulation model, while this paper constructs a feasible discrete rotating-chamber schedule and retains exact MIP bounds and proof mechanisms. The
 45 two problems are thus complementary rather than interchangeable.

An explicit mixed-integer linear programming (MILP) formulation is attractive for this scheduling problem because it exposes the physical assumptions and provides primal and dual bounds. The same fidelity, however,
 50 creates many assignment and resource-order binaries and weakly coupled timing variables. The resulting branch-and-cut search can be sensitive to which valid inequalities are retained at each separation round. Learning can guide this decision without replacing the solver’s feasibility and proof responsibilities (Bengio et al., 2021; Scavuzzo et al., 2024). HEM formulates cut retention
 55 as hierarchical cardinality and ordered-subset decisions (Wang et al., 2023b, 2024); nevertheless, its generic features do not identify how a cut distributes mass across variable types and objective-active continuous variables, greedy decoding commits to early choices, and an exact solve may spend substantial time before finding a competitive incumbent.

60 Taken together, the literature leaves an unresolved interface between an operationally faithful model for dual-source mixed-flow rotating-chamber equipment and a scalable exact solution mechanism that exploits the model’s structure. We address the modeling and computational gaps jointly: the first part of the framework describes the equipment through a two-stage MILP,
 65 and the second uses variable-type/objective-role information together with feasible schedule structure to enhance HEM within SCIP. The resulting logic is equipment physics $\rightarrow$ exact scheduling model $\rightarrow$ structural search bottleneck $\rightarrow$ learning-enhanced branch-and-cut.

The contributions are as follows.

- 70 1. A finite-horizon two-stage MILP is developed for dual-source mixed-flow rotating-chamber cluster tools. It jointly represents product and dummy-wafer flows, mixed $4 \times 1/2 \times 2$ recipes, rotary chamber events, ATR/VTR transfers, load-lock states, cleaning epochs, residency limits, and vacuum-zone dummy-wafer storage and reuse. Stage one minimizes makespan;

75 stage two performs makespan-preserving timing refinement conditional
on the retained discrete structure.

2. A structure-aware enhanced HEM method is developed for this MILP. A structure-preserving binary-skeleton (SPBS) warm start supplies an early feasible incumbent; a reconstructed 23-dimensional representation
80 exposes discrete and continuous variable roles; and A3C-inspired parallel policy training, bounded beam decoding, and role-submodular completion coordinate cut cardinality, membership, and order.
3. A paired computational study provides complementary evidence for the model, the completed algorithmic components, and manufacturing diffi-
85 culty. Small exact instances assess model validity; the complete nominal matrix compares seven solver configurations across independent solver and training seeds; controlled stability and factorial studies isolate the two-stage mechanism and manufacturing difficulty.

The remainder of the paper reviews the two research streams and posi-
90 tions the present study, formulates the rotating-chamber scheduling model, develops the structure-aware solution method, reports the computational study, and concludes with operational implications and research limitations.

2. Literature review and research positioning

2.1. Cluster-tool scheduling under mixed flows and auxiliary constraints

95 Cluster-tool scheduling has evolved from steady-state robot cycles toward models that account for product heterogeneity and auxiliary equipment constraints. Integrated patterns and parallel-chamber workload balance have been studied for dual-armed tools and multiple wafer types (Kim et al., 2021; Ko et al., 2021; Zhu et al., 2022). More recent work coordinates multi-type
100 concurrent processing or formulates two-wafer-type single-arm scheduling as a MIP (Li and Yang, 2025; Yang et al., 2025). These studies establish that release order, chamber allocation, robot tasks, and residency restrictions cannot be optimized independently when several product types share a tool.

Cleaning and other temporal restrictions add another coupling layer. Bi-
105 nary formulations and analytical methods have been developed for residency-constrained tools with cleaning (Qiao et al., 2022; Li et al., 2022; Lee et al., 2023). Purge, post-processing, and processing-time variation also change feasibility and cycle behavior (Yuan et al., 2023; Zhu et al., 2023; Pan et al.,

2024). At the equipment level, processing-module configuration and robot
 110 waiting must be coordinated (Wang et al., 2023a); multi-space modules in-
 troduce additional rotation and loading interactions (Zou et al., 2024; Gu
 et al., 2024). The dominant focus nevertheless remains a single material
 source, a cyclic regime, or a predetermined robot pattern. These assump-
 tions are restrictive for the finite-batch rotating-chamber system considered
 115 here, where product wafers and reusable vacuum-zone dummy wafers follow
 different routes before meeting in shared rotary modules.

2.2. Learning-based semiconductor and cluster-tool scheduling

Learning-based manufacturing scheduling increasingly combines a fast
 policy with problem-specific search. At the fab or production-system level,
 120 Chien and Lan (2021) integrate deep reinforcement learning with a hybrid
 genetic algorithm, while Zhang et al. (2024) use Q-learning to guide an
 estimation-of-distribution search. CNN-A3C has been used for semiconduc-
 tor scheduling under uncertain processing times and urgent orders (Liu et al.,
 2022); feedback and actor-critic control have likewise been combined to ex-
 125 ploit production-system knowledge (Li and Chang, 2022). Problem-specific
 encodings and greedy search remain relevant because learned policies alone
 do not guarantee effective local improvement (Hu et al., 2023).

At the cluster-tool level, DRL with adaptive search has been used for con-
 current release and robot-task decisions (Kim and Lee, 2025), and a related
 130 study addresses noncyclic scheduling directly (Kim and Kim, 2025). Hong
 et al. (2026) combine discrete-event simulation and DDPG to control contin-
 uous processing times in a PVD tool. The latter is the closest CIE study in
 equipment-level motivation, but it addresses a different control lever. Its pol-
 icy changes process duration in a simulated PVD environment; the present
 135 policy allocates valid cuts while an exact solver determines a discrete rotating-
 chamber schedule. This distinction motivates learning inside, rather than in
 place of, mathematical optimization.

2.3. Learning-enhanced exact optimization and research gap

MILP makes route, capacity, cleaning, and residency assumptions
 140 inspectable, although formulation choices can materially alter relaxations
 and search (Adelgren and Maravelias, 2023). Learning-enhanced exact
 optimization preserves this modeling role while guiding selected solver
 decisions (Bengio et al., 2021; Scavuzzo et al., 2024). Learned variable biases
 can support primal search (Khalil et al., 2022), and graph representations

145 can encode variable-constraint interactions (Cappart et al., 2023; Chen et al., 2023). SCIP provides the modular branch-and-cut interface used in this study (Achterberg, 2009; Bestuzheva et al., 2023).

Within branch-and-cut, cut selection has been learned through reinforcement learning, supervised ranking, and look-ahead imitation (Tang et al., 150 2020; Huang et al., 2022; Paulus et al., 2022). Instance-adaptive rules further demonstrate that a fixed scoring vector is not uniformly effective (Turner et al., 2023). HEM jointly predicts retained cardinality and an ordered cut subset (Wang et al., 2023b, 2024), while beam search provides a general mechanism for retaining alternative partial sequences (Choo et al., 2022). 155 These methods establish the algorithmic ingredients, but they do not connect an application-feasible incumbent and variable-type and objective roles to hierarchical cut decoding.

Table 1: Positioning relative to representative recent cluster-tool studies.

Study	Setting	Primary decision layer	Method
Qiao et al. (2022)	Cleaning-constrained CT	Residency and cleaning schedule	Binary IP
Gu et al. (2024)	Multi-space CT	Timing of preset robot sequences	LP
Yang et al. (2025)	Two-type CT	Release, robot tasks, and residency	MIP
Kim and Lee (2025)	Concurrent CT	Wafer release and robot sequence	DRL + search
Hong et al. (2026)	PVD CT	Continuous process-time control	DES + DDPG
Present study	Dual-source rotating-chamber CT	Mixed-flow assignment, resources, and event times	Two-stage MILP + learned B&C

Table 1 exposes the intersection left open by the two research streams. At the modeling level, the reviewed formulations do not jointly cover finite- 160 horizon dual-source flows, mixed rotary recipes, cleaning segmentation, load-lock and robot conflicts, and reusable vacuum-zone dummy wafers. At the algorithmic level, existing learning methods either act directly on schedules or use generic MIP representations; they do not combine a physically feasible

warm start, variable-role reconstruction, hierarchical policy training, beam
 165 decoding, and structural set completion. The present model and RDMCT-
 HBS address these two gaps within one solver-compatible framework.

3. Dual-source mixed-flow rotating-chamber system and two-stage MILP formulation

3.1. Equipment architecture and assumptions

170 The investigated equipment is a finite-batch rotating-chamber cluster tool
 with separate atmospheric and vacuum domains. Product wafers follow the
 route

$$\begin{aligned} \text{LP} &\rightarrow \text{ATR} \rightarrow \text{AL} \rightarrow \text{ATR} \rightarrow \text{LL}_U \rightarrow \text{VTR} \\ &\rightarrow \text{CH} \rightarrow \text{VTR} \rightarrow \text{LL}_L \rightarrow \text{ATR} \rightarrow \text{LP}. \end{aligned}$$

where LP denotes a load port, ATR and VTR are the atmospheric and vac-
 uum transfer robots, AL is an aligner, and LL_U and LL_L are the upper and
 175 lower load locks. Vacuum-zone dummy wafers travel from the vacuum-zone
 dummy-wafer storage area to the shared process chambers through the VTR.
 Chambers CH2 and CH3 rotate and share transfer and loading resources.

Two processing modes are present. A 4×1 rotary batch contains two
 physical sides; the front side is loaded first and unloaded last, while the back
 180 side is loaded second and unloaded first. A product-deficient tail side is filled
 by a pair of dummy wafers. In 2×2 mode, product-wafer pairs form an
 ordered chain bounded by leading and trailing dummy-wafer pairs, and each
 product pair participates in two adjacent exposures. A cleaning boundary
 splits the chain at an exposure boundary and converts the chamber to a
 185 dummy-wafer-only state. Each vacuum-zone dummy wafer is reusable only
 after its individual load-to-unload occupancy intervals end. Robot moves
 and incompatible loading operations are non-overlapping.

Processing and transfer times are deterministic, all wafers are available
 at the beginning of the finite horizon, and preemption is not allowed. Each
 190 activated operation is assigned to exactly one compatible chamber or slot
 group. The model optimizes assignments, mode choices, cleaning epochs,
 and resource orders jointly.

3.2. Sets, parameters, and variables

Let $\mathcal{W}$ be the product wafers and $\mathcal{P}^4$ and $\mathcal{P}^2$ the product-pair sets pre-
 195 scribed for the two recipe families. Let $\mathcal{C} = \{\text{CH2}, \text{CH3}\}$, $\mathcal{B}_c$ be candidate

4×1 batches, $\mathcal{Q}_c$ be 2×2 chain positions, $\mathcal{E}_c$ be cleaning-separated process epochs, and $\mathcal{T}_c$ be the reusable vacuum-zone dummy wafers assigned to chamber c . The complete event set is $\mathcal{O}$, the physical resources are $\mathcal{R}$, and $\mathcal{A}$ contains technological precedence arcs.

200 Binary x_{pcbh}^4 assigns $p \in \mathcal{P}^4$ to chamber c , batch b , and side $h \in \{F, B\}$; x_{pcq}^2 assigns $p \in \mathcal{P}^2$ to chain position q . Variables u_{cb}^4 and u_{cq}^2 activate batches and chain positions, a_{vce} assigns process cycle v to epoch e , and $g_{j\tau}$ assigns dummy-wafer operation j to physical dummy wafer τ . Binary $y_{oo'}^r$ orders two actions on shared resource r . Every action o has start s_o , completion
205 $e_o = s_o + d_o$, and $C_{\max}$ is the latest product return to an LP.

3.3. Stage-one makespan model

Every prescribed product pair is assigned once:

$$\sum_{c \in \mathcal{C}} \sum_{b \in \mathcal{B}_c} \sum_{h \in \{F, B\}} x_{pcbh}^4 = 1 \quad p \in \mathcal{P}^4, \quad (1)$$

$$\sum_{c \in \mathcal{C}} \sum_{q \in \mathcal{Q}_c} x_{pcq}^2 = 1 \quad p \in \mathcal{P}^2. \quad (2)$$

With filler indicator f_{cbh} , active 4×1 sides and contiguous batch/chain positions satisfy

$$\sum_{p \in \mathcal{P}^4} x_{pcbh}^4 + f_{cbh} = u_{cb}^4, \quad (3)$$

$$u_{c,b+1}^4 \leq u_{cb}^4, \quad u_{c,q+1}^2 \leq u_{cq}^2. \quad (4)$$

210 Additional tail equalities permit $f_{cbh} = 1$ only on the last active batch.

For every action, $e_o = s_o + d_o$. Product and dummy-wafer routes impose

$$s_{o'} \geq e_o + \delta_{oo'}, \quad (o, o') \in \mathcal{A}, \quad (5)$$

where $\delta_{oo'}$ includes rotation, pressure transition, or mandatory empty travel. Two actions sharing unit-capacity resource r obey

$$s_{o'} \geq e_o + \rho_{oo'}^r - M(1 - y_{oo'}^r), \quad (6)$$

$$s_o \geq e_{o'} + \rho_{o'o}^r - M y_{o'o}^r, \quad o, o' \in \mathcal{O}_r. \quad (7)$$

This form covers ATR, AL, VTR, stateful load-lock slots, and chamber oc-
215 cupancy. Endpoint-dependent ρ^r accounts for robot repositioning.

Rotary equalities enforce front load, rotation, back load, immediate process, back unload, rotation, and front unload for each 4×1 batch. In 2×2 , adjacent exposures share the intermediate product pair and a bridge action replaces the outgoing pair by the incoming pair. All active 4×1 batches on a chamber precede its 2×2 block.

Let H be the maximum process cycles between cleans and let z_{ce} activate epoch e . Process-cycle assignment and cleaning capacity satisfy

$$\sum_{e \in \mathcal{E}_c} a_{vce} = u_{vc}, \quad (8)$$

$$\sum_v a_{vce} \leq H z_{ce}, \quad (9)$$

$$s_{c,e+1}^{\text{clean}} \geq E_{ce}^{\text{last}}, \quad (10)$$

where E_{ce}^{last} is a linear completion envelope. Linking constraints insert a dummy-wafer-only cleaning batch between consecutive epochs and allow a 2×2 chain to split only at an exposure boundary.

Each dummy-wafer operation selects one chamber-bound reusable dummy wafer,

$$\sum_{\tau \in \mathcal{T}_c} g_{j\tau} = 1, \quad j \in \mathcal{J}_c^{\text{DW}}, \quad (11)$$

and occupancy intervals assigned to the same physical dummy wafer τ are disjoint by Eq. (7). Product and robot residency limits have the form

$$0 \leq s_{\text{out}(w,r)} - e_{\text{in}(w,r)} \leq \bar{R}_{wr}. \quad (12)$$

Finally,

$$C_{\max} \geq e_w^{\text{return}} \quad (w \in \mathcal{W}), \quad \min_{z \in \mathcal{F}} C_{\max}(z), \quad (13)$$

so the primary objective covers the full route back to the LP. The feasible set $\mathcal{F}$ contains the assignment, route, resource, rotation, cleaning, residency, load-lock-state, and dummy-wafer availability constraints.

3.4. Makespan-preserving conditional refinement

Several membership, slot, and alias variables are fixed to zero or one by mode-induced equalities. Declaring such variables continuous while retaining their fixing equalities removes redundant integrality declarations without changing their values or the projection of $\mathcal{F}$. Genuine assignment, mode, ordering, and cleaning decisions remain integer.

240 Let $\hat{z}$ be the best stage-one solution, $\hat{C} = C_{\max}(\hat{z})$, and $\mathcal{D}_{\text{fix}}$ the discrete decisions retained in the second stage. Let $\varepsilon_C \geq 0$ denote the makespan-preservation tolerance. The refinement imposes

$$z_d = \hat{z}_d \quad (d \in \mathcal{D}_{\text{fix}}), \quad C_{\max}(z) \leq \hat{C} + \varepsilon_C, \quad (14)$$

while selected double-gripper and continuous timing decisions remain free. It minimizes

$$S_{\text{lin}}(z) = \alpha_w \sum_{i \in \mathcal{W}} w_i + \alpha_m w^{\max} + \alpha_c \sum_{j \in \mathcal{C}} \delta_j + \alpha_e \sum_{k \in \mathcal{E}} e_k, \quad (15)$$

245 where w_i are waiting or idle quantities, $w^{\max}$ is the largest individual wait, δ_j are absolute deviations between successive start cadences, and e_k are soft wait-cap excesses. The weights are fixed across all compared methods. If stage one proves optimality, Eq. (14) preserves the globally optimal makespan. If stage one terminates with a feasible incumbent but without a proof, stage
250 two preserves that incumbent makespan; its stability optimum is conditional on the fixed discrete subspace.

3.5. Model scope and induced computational structure

The formulation differs from sequence-specific steady-state LPs in both horizon and decision scope. It optimizes a finite batch, treats product wafers
255 and vacuum-zone dummy wafers as interacting material streams, and determines grouping, assignment, cleaning, transport order, and event times within one model. The second stage does not claim a global lexicographic optimum over every stage-one-optimal discrete schedule: it refines timing conditional on the discrete structure retained from stage one. This distinction
260 protects the primary makespan while keeping the refinement computationally interpretable.

Operational fidelity is also the source of computational difficulty. Assignment, activation, cleaning-epoch, dummy-wafer, and resource-order binaries interact with several classes of continuous timing variables. Consequently,
265 two cuts with similar generic efficacy may constrain very different physical parts of the schedule. The next section uses this variable-type/objective-role information to design the warm start, cut representation, and decoding procedure.

4. Structure-aware enhanced HEM for exact solution

270 4.1. Integrated solution architecture

The proposed method targets the two principal symptoms of the formulation: late incumbent discovery and structurally heterogeneous cut pools. The warm start addresses the primal trajectory, whereas the enhanced HEM policy allocates the cut budget using information exposed by the formulation and solver metadata. These mechanisms remain subordinate to the exact
275 solver and therefore do not alter the feasible region.

The solution sequence is

SPBS incumbent $\rightarrow$ hierarchical cut policy $\rightarrow$ beam/role completion
 $\rightarrow$ SCIP branch-and-cut $\rightarrow$ two-stage schedule.

SPBS and learned cut selection target different parts of the primal-dual trajectory. SPBS seeks an early feasible incumbent, whereas the learned selector
280 allocates a limited cut budget during separation. In the principal method comparison every algorithm receives the same precomputed SPBS solution. End-to-end time includes loading and injecting that solution, all callback work, and SCIP solving. This common-start design holds the primal-start mechanism fixed in the nominal comparison so that the compared configurations
285 differ in cut selection and decoding.

4.2. Structure-preserving binary-skeleton warm start

Directly fixing a complete schedule can impose a brittle heuristic path. SPBS instead fixes a small interpretable binary skeleton in a temporary model and uses SCIP to complete the remaining timing and resource decisions. For
290 each binary variable x_j , it constructs a one-hot role vector

$$z_j \in \{0, 1\}^{|\mathcal{H}_s|}, \quad \mathcal{H}_s = \{A, U, E, L, R_\ell, R_h, O\},$$

where A denotes wafer or batch assignment, U activation, E cleaning or process epochs, L load-lock slots, R_ℓ low-risk local orders, R_h high-impact VTR orders, and O residual binaries.

Each role has a confidence and a fixing rule. Assignment, activation, cleaning-epoch, and slot variables have high confidence because they follow
295 from the recipe grammar. ATR, aligner, and load-lock orders have intermediate confidence. VTR order is more influential and is released in the relaxed

SPBS profile. For temporary model $\widetilde{M}$, a skeleton value $\widehat{x}_j$ is imposed only when the role/profile rule selects it:

$$\ell_j = u_j = \widehat{x}_j \quad \text{if} \quad \pi(z_j, \kappa_{h(j)}, \text{profile}) = \text{fix}.$$

300 Continuous event times, waiting variables, and unfixed resource orders remain free.

The skeleton follows the recipe physics. Complete 4×1 rotary batches are filled first. In 2×2 mode, product pairs form contiguous exchange chains bounded by dummy-wafer events. When a chain reaches a cleaning trigger, 305 the next admissible event converts the chamber to a dummy-wafer-only state before cleaning, after which product exchange resumes. A feasible completion is passed to the original model through SCIP’s primal-solution interface. No SPBS constraint is added to the original solve, so its feasible set and proof requirements are unchanged.

310 4.3. Candidate interface and reconstructed 23D features

At separation round t , SCIP supplies a set $\mathcal{C}_t$ of valid candidate inequalities. An efficacy prefilter retains at most 128 candidates and the selector returns no more than 16. Cut coefficients and right-hand sides are never modified.

315 The adapted HEM representation contains 13 generic measurements: objective parallelism, efficacy, support, integral support, normalized violation, and four statistics each for cut and objective coefficients. To reconstruct the interaction between cut geometry and variable-type roles, define

$$\mathcal{G} = \{B, I, C_{\text{obj}}, C_{\text{aux}}, O\},$$

for binary, general-integer, objective-active continuous, continuous auxiliary, 320 and unclassified variables. For candidate row $\sum_j a_{ij}x_j \leq b_i$, its coefficient mass on role g is

$$p_{ig} = \frac{\sum_{j: x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \varepsilon_{\text{num}}}. \quad (16)$$

Here $\varepsilon_{\text{num}} > 0$ is a numerical stabilizer. Role assignment uses SCIP variable metadata and is invariant to variable renaming. Table 2 lists the ten appended role-aware features.

325 The coupling term is zero for a cut supported entirely on one side of the discrete–continuous split and is largest for balanced support. Dominant mass and entropy distinguish focused cuts from cuts mixing several roles. Together with the generic features, the candidate representation has 23 dimensions.

Table 2: Ten role-aware features appended to the 13 generic cut features.

Count	Reconstructed features
5	Coefficient mass on B , I , C_{obj} , C_{aux} , and O
1	Mass on variables with finite global lower and upper bounds
1	Positive-coefficient mass
1	Discrete-continuous coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$
1	Largest non-residual role mass
1	Normalized role entropy $-\sum_g p_{ig} \log p_{ig} / \log \mathcal{G} $

4.4. A3C-inspired parallel hierarchical policy gradient

330 The hierarchical pointer policy (Vinyals et al., 2015; Wang et al., 2023b) first predicts retained cardinality k_t and then an ordered cut sequence:

$$\pi_\theta(a_t | s_t) = \pi_{\theta_c}^{\text{count}}(k_t | s_t) \prod_{\ell=1}^{k_t} \pi_{\theta_p}^{\text{ptr}}(i_\ell | s_t, i_{1:\ell-1}). \quad (17)$$

Both policies use a 128-dimensional hidden representation and one pointer glimpse. The reward is the negative SCIP-reported primal-dual integral,

$$R = -\text{PDI}_{\text{SCIP}}, \quad (18)$$

so larger reward represents a better anytime primal-dual trajectory.

335 Independent worker processes evaluate complete SCIP trajectories in parallel. Their policy gradients are aggregated before a centralized update. With exponential moving-average baseline

$$b_{q+1} = \beta b_q + (1 - \beta) \bar{R}_q, \quad (19)$$

the estimator is

$$\widehat{\nabla_\theta J} = \frac{1}{B} \sum_{\ell=1}^B (R_\ell - b_q) \nabla_\theta \log \pi_\theta(a_\ell | s_\ell). \quad (20)$$

340 This organization follows the parallel exploration and shared-policy principle of A3C (Mnih et al., 2016). The actual estimator in Eq. (20) is specified by the centralized update and EMA control variate; throughout the paper it is termed A3C-inspired parallel hierarchical policy gradient.

4.5. Beam decoding and role-submodular completion

Training samples ordered subsets stochastically. At evaluation, the
 345 adapted HEM baseline uses greedy pointer decoding, whereas the beam
 variants retain the three highest-scoring partial sequences. End-of-sequence
 handling, candidate pools, and cut caps are identical across decoding modes.
 Beam and callback computation are charged to the end-to-end limit.

The neural order can concentrate its budget on one structural pattern.
 350 The first $\lceil \gamma k_t \rceil$ candidates are therefore retained as anchors A_t , and an ex-
 pansion pool R_t of at most αk_t candidates is formed by appending rows with
 high normalized structural quality. For $S \subseteq R_t$, define

$$\begin{aligned}
 F_t(S) = & \sum_{i \in S} (0.35q_i + 0.20r_i) + 0.25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} \\
 & + \frac{0.20}{|R_t|} \sum_{u \in R_t} \max_{i \in S} s_{ui},
 \end{aligned} \tag{21}$$

where q_i is normalized structural quality, r_i the neural-rank prior, d_g role
 demand in R_t , and $s_{ui} \in [0, 1]$ clipped cosine similarity between generic-role
 355 descriptors. The first term preserves quality and neural order, the concave
 term gives diminishing returns to repeated role mass, and the facility-location
 term rewards representation of the candidate pool.

Starting from A_t , the method repeatedly adds the candidate with largest
 marginal gain until k_t cuts are selected. For fixed anchors and expansion
 360 pool, the nonnegative modular, concave-over-modular, and facility-location
 components are monotone submodular. Cardinality-constrained greedy com-
 pletion therefore obtains at least a $(1 - 1/e)$ fraction of the best residual gain
 inside R_t (Nemhauser et al., 1978). This bound concerns residual set comple-
 tion, not branch-and-cut runtime or final MIP quality. The implementation
 365 uses $\alpha = 2$ and $\gamma = 0.25$.

4.6. Solver validity and computational accounting

RDMCT-HBS ranks only inequalities already generated and validated by
 SCIP. It does not generate constraints, modify coefficients, prune nodes, or
 declare infeasibility. Unselected candidates remain available according to the
 370 solver interface. Thus the learned component can change search order and
 runtime but not the scheduling feasible region or the validity of an eventual
 proof. Feature computation, device transfer, policy inference, beam decoding,

role completion, SPBS loading, and SCIP solution time are included in end-to-end time. The common SPBS solution is loaded before each nominal solve; its construction time is outside the nominal online timing.

5. Computational study

5.1. Experimental protocol

5.1.1. Research questions

The experiments address four questions:

- RQ1. Does the two-stage MILP generate physically feasible schedules and preserve the stage-one makespan while improving timing stability?
- RQ2. Does the integrated cut selector improve anytime performance over SCIP, adaptive cut selection, and adapted HEM?
- RQ3. What are the separate effects of SPBS, reconstructed features, role-submodular completion, and beam decoding?
- RQ4. How do wafer count, recipe mix, and processing-time scale affect computational difficulty?

5.1.2. Instances and evaluation suites

Table 3 summarizes the completed evaluation suites reported in this paper.

The nominal suite is the principal algorithm-comparison set. The SPBS suite isolates the warm-start main effect under default SCIP. Factorial instances vary manufacturing conditions jointly, whereas the sensitivity suite perturbs motion time, processing time, cleaning frequency, and vacuum-zone dummy-wafer inventory one factor at a time. Training and evaluation instances are separated by experimental role.

5.1.3. Compared methods and ablations

Table 4 defines the seven configurations in the nominal comparison.

The adaptive selector uses validation-chosen weights. For direction-to-cutoff distance, efficacy, integer support, and objective parallelism, respectively, the fixed values are 0.25, 0, 0.50, and 0.25. They remain fixed for all nominal observations.

Table 3: Completed evaluation suites.

Suite	Design	Conditions
Verification	Four small pure and mixed-flow instances	4 instances
Nominal main	20 instances, five solver seeds, and five training seeds	2700 runs
Controlled stability	20 instances, three profiles, five solver seeds, and five training seeds	1500 runs
Factorial	Wafers {8, 16, 24, 32}, five recipe ratios, and processing scale {0.9, 1.0, 1.1}	300 runs
SPBS	48 mixed-flow instances, automatic versus no warm start, and ten solver seeds	960 runs
Parameter sensitivity	Eight one-factor-at-a-time perturbations with ten solver seeds	80 runs

Table 4: Methods in the nominal comparison.

Method	Configuration
SCIP	Default SCIP cut selection
Adaptive cut selection	Deterministic four-parameter hybrid selector
HEM-13D	Adapted hierarchical count/pointer policy with greedy decoding
HEM-13D+Beam	Same HEM-13D model with beam width three
23D Features Only	Reconstructed features without role completion
Structure+Greedy	23D policy and role completion with greedy decoding
RDMCT-HBS	23D policy, role completion, and beam width three

All nominal methods use the same MIP, instance, solver seed, global SCIP settings, candidate caps, resource limits, and automatic SPBS solution. Learned methods use the corresponding pre-specified model realization. HEM-13D+Beam changes only decoding relative to HEM-13D. Structure+Greedy and RDMCT-HBS use the same structural policy and differ only in greedy versus beam decoding.

5.1.4. Training and computational environment

Each policy configuration is trained with five independent random seeds for 60 epochs. The lower-level policy uses Adam with learning rate 10^{-4} , batch size eight, maximum gradient norm two, and eight training samples per epoch. The high-level count policy uses learning rate 5×10^{-4} . Two sample processes collect trajectories in parallel. The learning rate is multiplied by 0.96 every ten epochs. Candidate and selected-cut caps are 128 and 16.

Experiments use SCIP 10.0.1 through PySCIPOpt 6.1.0 and Python 3.12.12. Policy inference uses PyTorch 2.11.0 with CUDA 13.0 on an NVIDIA GeForce RTX 5060 Ti. The host runs Windows 11 on an Intel Core Ultra 7 265KF with 20 physical cores and 31.7 GB RAM. Nominal, factorial, SPBS, and sensitivity solves use a 600-s wall-clock limit and a 2048-MB SCIP memory limit. Each solver invocation is single-threaded.

5.1.5. Performance measures and statistical analysis

The primary measure is the SCIP primal-dual integral (PDI); lower values represent faster joint improvement of primal and dual bounds. Secondary measures are incumbent availability, final primal bound, final relative gap, optimal-solution rate, nodes, callback overhead, stage-specific status, stability measures, and end-to-end time. A time-limited solve remains informative through its incumbent, bound, gap, and PDI. For every saved stability-study schedule, physical route waiting is reconstructed from six nonnegative event gaps per product wafer: before alignment, before upper-load-lock transfer, after the upper load lock, before processing, after processing, and after the lower load lock. Their sum and maximum define route-total and route-maximum wait. Cadence CV and the sum of absolute consecutive cadence changes are reconstructed from the start times of active chamber batches within each cleaning epoch. This event-time definition is identical for all three stability profiles and excludes secondary-objective slack variables.

Comparisons are matched by manufacturing instance and solver seed and, for learned methods, training seed. Repeats are first aggregated within a man-

ufacturing instance so that the manufacturing instance is the statistical unit.
 440 Principal comparisons include paired differences, bootstrap 95% confidence
 intervals, two-sided Wilcoxon signed-rank tests where applicable, and Holm-
 adjusted p -values. Completed component contrasts report matched mean
 differences and Holm-adjusted p -values. Incumbent and failure rates are re-
 445 ported alongside continuous metrics. This instance-centered design treats
 heterogeneous MIP cases, rather than repeated solver seeds, as the units of
 generalization (Gleixner et al., 2021).

5.2. Model verification and two-stage behavior

The 81 generated formulations—60 core, 9 larger-scale, 8 sensitivity, and
 4 verification instances—were parsed to measure structural growth. Table 5
 450 summarizes the 60 core and 9 larger-scale instances and shows that the largest
 core formulation contains 13,663 variables and 77,746 constraints, while the
 largest 64-wafer formulation contains 44,323 variables, 38,135 binary vari-
 ables, and 276,959 constraints. These counts characterize formulation scale
 independently of solver performance.

Table 5: Structural scale of the generated two-stage MIPs.

Group/wafers	Instances	Mean variables	Max binaries	Max constraints
Core/8	15	1163.6	900	3652
Core/16	15	2645.2	2401	13441
Core/24	15	6711.2	7500	37554
Core/32	15	9984.0	11145	77746
Large/40	3	15796.3	15113	88928
Large/48	3	21610.0	19548	129223
Large/64	3	36475.0	38135	276959

455 Table 6 reports four pure and mixed-flow verification instances. SCIP
 found an optimal stage-one schedule and an optimal conditional stability
 refinement for every instance. The wait columns use the same six event-
 reconstructed product-route intervals used throughout the study. The re-
 sulting schedules satisfied the active route, assignment, resource, cleaning,
 460 residency, and dummy-wafer availability constraints in an independent feasi-
 bility check.

The increasing node count from one to 18,195 illustrates the rapid growth
 of the coupled assignment and resource-order search even at small scale. The

Table 6: Two-stage verification results with event-reconstructed route waits.

Configuration	Products	$C_{\max}$	Route-total wait	Route-maximum wait	Nodes	Time (s)
Pure 4×1	4	458.0	380.0	76.0	1	0.009
Pure 2×2	4	584.0	112.0	28.0	54	0.039
Mixed flow	8	834.0	506.0	76.0	504	0.458
Mixed flow	16	1170.0	1550.0	99.0	18195	24.390

zero final gaps and independent checks confirm that the formulation exactly
 465 represents the tested routing, resource, cleaning, residency, and dummy-wafer
 constraints. Algorithmic performance is assessed separately.

5.3. Nominal performance across training realizations

Table 7 reports the complete nominal comparison.

Table 7: Nominal comparison across independent training realizations. SCIP and adaptive cut selection have 100 runs each; learned methods have 500 runs each. Statistics are first averaged within each of 20 manufacturing instances. Gap is reported in percent.

Method	PDI	Gap (%)	Solve time (s)	Optimal (%)	Holm p
SCIP	16919.01	119.54	226.05	64.0	1.000
Adaptive cut selection	16854.87	127.18	222.34	62.0	1.000
HEM-13D	16823.18	127.24	219.65	62.4	1.000
HEM-13D+Beam	16829.59	127.19	219.95	62.4	1.000
23D Features Only	16620.91	125.33	219.42	62.6	1.000
Structure+Greedy	16598.11	126.03	218.44	63.2	1.000
RDMCT-HBS	16585.70	126.01	218.32	63.2	Reference

Across five training realizations, the mean PDI of RDMCT-HBS is 237.49
 470 (1.41%) below HEM-13D and 333.31 (1.97%) below SCIP. At the instance
 level, it is lower than HEM-13D on 14 of 20 instances and lower than SCIP
 on 8. Bootstrap intervals for the comparator-minus-RDMCT-HBS differ-
 ences are $[-446.70, 846.26]$ and $[-382.16, 999.70]$. Both include zero, and
 every global Holm-adjusted PDI comparison equals 1.000. Thus, these in-
 475 tervals and adjusted tests indicate heterogeneous instance-level gains. The
 complete benchmark nevertheless establishes RDMCT-HBS as the strongest
 mean anytime performer under the evaluated protocol: it has the lowest mean
 PDI and solve time among the seven methods and returns an incumbent in
 every run. SCIP retains a lower mean final gap and a slightly higher optimal
 480 rate. Independent SCIP reconstruction verifies all 2700 saved incumbents
 using the solver feasibility tolerance plus a 10^{-12} text-serialization allowance.

5.4. Component analysis

Table 8 reports the completed pre-specified component contrasts.

Table 8: Completed pre-specified component contrasts. Negative differences favor the first configuration named in a row.

Contrast		Component isolated	Δ PDI	Holm p
HEM-13D+Beam	vs.	Beam without reconstructed features	+6.40	1.000
HEM-13D				
23D Features Only	vs.	Feature reconstruction	−202.27	1.000
HEM-13D				
Structure+Greedy	vs.	23D Role completion	−22.80	1.000
Features Only				
RDMCT-HBS	vs.	Struc- Beam with role completion	−12.42	1.000
ture+Greedy				

Feature reconstruction has the largest observed mean PDI reduction among the component contrasts (202.27). The observed mean differences for role completion and structure-conditioned beam decoding are −22.80 and −12.42, respectively. The observed beam difference changes sign to +6.40 in the generic 13D representation. No individual component contrast reaches Holm-adjusted significance. The controlled means nevertheless identify feature reconstruction as the largest contributor in the evaluated component chain and show that the observed beam effect depends on the structure-aware representation.

5.5. Independent effect of the SPBS warm start

The 960-run SPBS experiment crosses 48 mixed-flow instances, ten solver seeds, and automatic versus no warm start under otherwise identical default-SCIP settings. Table 9 reports instance-first averages.

Table 9: Independent SPBS warm-start comparison.

Warm start	Runs	Incumbent (%)	Optimal (%)	PDI
Automatic SPBS	480	100.00	54.58	20278.78
None	480	51.67	51.67	27473.05

SPBS increases mean incumbent availability by 48.33 percentage points (95% CI [34.58, 62.29]; Holm $p = 2.55 \times 10^{-5}$) and reduces mean PDI by

7194.27, or 26.19% (95% CI [5145.78, 9317.61]; Holm $p = 3.46 \times 10^{-8}$). The
500 optimal-rate increase of 2.92 percentage points and the 2.70-s mean time
reduction do not remain significant after Holm adjustment. All 960 runs are
retained; the 728 saved incumbents pass independent feasibility validation.
This experiment identifies the SPBS main effect under default SCIP and does
not claim an interaction with the learned cut selector. Offline construction
505 time is excluded in both arms; online loading and solver time are included.

5.6. Controlled evaluation of two-stage schedule refinement

The 1500-run controlled study crosses 20 nominal instances, five solver
seeds, five training seeds, and three profiles. The full profile enables the
second stage with the proposed linear waiting-cadence objective; *stage2-off*
510 disables refinement; and *linear-off* retains the second stage but uses the
quadratic legacy objective. Each profile contains 500 runs and returns an
incumbent in every run. Table 10 reports instance-first averages.

Table 10: Controlled two-stage stability study. Route waits and cadence statistics are
reconstructed from physical event times in the saved feasible schedules; waits are in model
time units.

Profile	Route-total wait	Route-maximum wait	Cadence CV	Cadence deviation
Full	2596.01	296.33	0.130	22.04
Linear-off	2469.47	293.27	0.134	60.18
Stage2-off	5398.60	952.57	0.216	370.41

Relative to disabling stage two, the full formulation reduces mean phys-
ical route-total wait by 2802.58 (51.91%; 95% CI [724.30, 5836.32]; Holm
515 $p < 0.00004$) and route-maximum wait by 656.24 (68.89%; 95% CI [52.00,
1536.37]; Holm $p = 0.00224$). Cadence CV falls by 39.70% (Holm $p = 0.0214$),
and cadence-deviation sum falls by 94.05% (Holm $p = 0.0184$). All differ-
ences are computed after aggregating the 25 matched solver/training repe-
titions within each of the 20 manufacturing instances. Compared with the
520 quadratic second-stage objective, the proposed linear objective further re-
duces cadence-deviation sum by 63.38% (95% CI [4.52, 94.68]; Holm $p =$
0.0377); its route-total wait, route-maximum wait, and cadence CV dif-
ferences are not multiplicity-adjusted significant. These results establish
that conditional second-stage refinement produces materially tighter prod-
525 uct routes and more regular processing cadence under the common 600-s
end-to-end budget.

5.7. Manufacturing-factor effects

The full $4 \times 5 \times 3$ factorial design contains 60 instances and five solver seeds per cell (300 runs), with no execution failure and an incumbent in every run. Table 11 shows the steep scale transition: all 8- and 16-wafer runs are solved to optimality, whereas the optimal rate falls to 36% and 20% at 24 and 32 wafers.

Table 11: Marginal computational difficulty by wafer count in the factorial study.

Wafers	Runs	Optimal (%)	PDI	Time (s)
8	75	100	28.13	0.34
16	75	100	1876.54	33.86
24	75	36	31155.94	408.55
32	75	20	32959.89	433.20

The prespecified interaction model ($R^2 = 0.998$ for PDI) identifies wafer count and recipe composition as the dominant joint source of difficulty. At the 1:1 recipe mix, the additional PDI interaction is 37729.48 for 24 wafers (95% CI [34486.94, 40972.01]) and 40788.63 for 32 wafers (95% CI [37546.09, 44031.16]). By contrast, the main-effect intervals for processing time scale cross zero. All prespecified processing-scale main-effect and interaction confidence intervals cross zero. Within this factorial design, the supported difficulty transition is therefore associated with wafer-count– recipe coupling rather than proportional processing-duration changes.

Figure 1 reports every prespecified PDI coefficient with its 95% confidence interval, making the scale–recipe interactions and the processing-scale main and interaction effects whose intervals include zero directly comparable.

5.8. One-factor-at-a-time parameter sensitivity

The 80-run sensitivity study perturbs a 24-wafer 1:1 mixed-flow instance through motion-time scales of 0.8 and 1.2, processing-time scales of 0.8 and 1.2, cleaning intervals of 3 and 10, and vacuum-zone dummy-wafer inventory of 10, with the nominal setting retained as a reference. Each of the eight settings uses ten solver seeds. Every run completes without an execution failure, returns an incumbent, and passes independent feasibility validation; 20 of 80 runs prove optimality. Across settings, mean PDI ranges from 28899.41 at cleaning interval 10 to 42478.32 at motion scale 1.2. This one-method

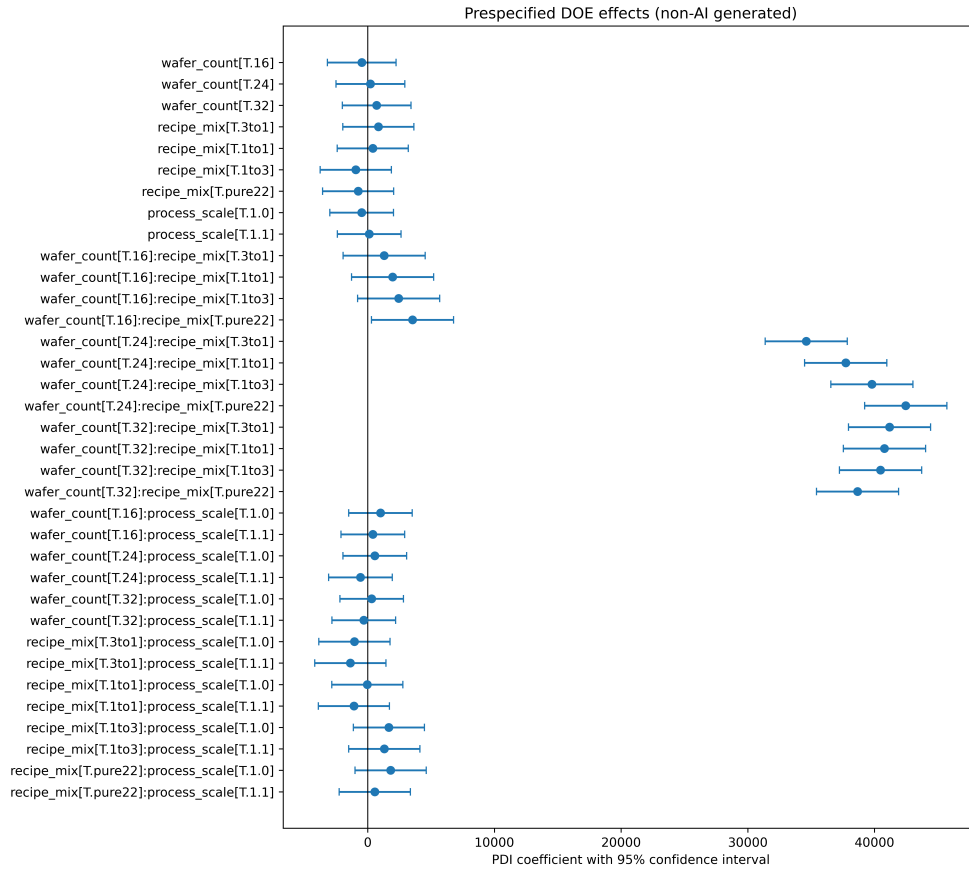


Figure 1: Prespecified factorial effects on PDI. Points are OLS coefficients and bars are 95% confidence intervals; the vertical line denotes zero. The figure is generated directly from the complete 300-run DOE.

study shows that the formulation and SPBS-assisted exact solve remain operational through the prescribed parameter perturbations; it is not used to infer a cross-method robustness ranking.

5.9. Operational interpretation and limitations

The proposed framework separates three responsibilities. Manufacturing engineers specify verifiable route, chamber, cleaning, and dummy-wafer constraints; SCIP generates valid inequalities and retains responsibility for mathematical proof; and the learned component allocates a limited cut budget. This separation is valuable when schedule feasibility cannot be delegated to a black-box predictor. The two-stage model also separates throughput-oriented makespan from operational smoothness: the second stage is designed to reduce avoidable waiting and cadence variation without sacrificing the retained stage-one makespan.

In the complete nominal matrix, RDMCT-HBS leads all seven configurations in mean PDI and solve time under the evaluated protocol. Its nominal instance-level intervals include zero, indicating that the gain varies across instances. Independently, the SPBS experiment establishes large, multiplicity-adjusted improvements in incumbent availability and PDI, while the controlled two-stage study provides multiplicity-adjusted evidence for the scheduling mechanism. Wall-clock comparisons are hardware-dependent, and feature extraction, SPBS loading, and beam decoding remain in the common online time budget. The role representation depends on the variable taxonomy exposed by SCIP, and the $(1 - 1/e)$ result applies only to residual completion inside a fixed expansion pool. The model assumes deterministic processing and transfer times; equipment failures, online arrivals, rescheduling, and fab-level interactions remain outside its present scope. Recent module-failure scheduling work further indicates that equipment-state changes warrant a dedicated extension rather than an untested robustness claim (Zhu et al., 2026).

6. Conclusions

This paper addresses finite-horizon scheduling of a dual-source mixed-flow rotating-chamber cluster tool through an operationally explicit two-stage MILP. The formulation connects product and dummy-wafer routes with mixed $4 \times 1/2 \times 2$ recipes, rotary chamber events, robotic transfers, load-lock states, cleaning, residency, and reusable vacuum-zone dummy wafers.

Makespan is optimized first, after which schedule timing is refined conditional on the retained discrete structure and primary objective.

The computational contribution is a structure-aware enhancement of HEM. Its central idea is to exploit information induced by the scheduling model at two points in the exact solve: SPBS supplies a physically structured incumbent, and variable-role features distinguish how candidate cuts act on discrete choices and continuous timing. A3C-inspired parallel policy-gradient training, bounded beam decoding, and role-submodular completion then allocate the cut budget, while SCIP retains feasibility and proof responsibilities.

Four verification instances reach zero gap in both stages and satisfy the independently checked physical constraints. Across the complete 2700-run nominal matrix, RDMCT-HBS records the lowest observed mean PDI and solve time, with PDI reductions of 1.41% relative to adapted HEM and 1.97% relative to SCIP, thereby establishing the strongest mean anytime performance under the evaluated nominal protocol. The instance-level intervals show that the magnitude of this gain is heterogeneous. Feature reconstruction has the largest observed mean component reduction, while role completion and structure-conditioned beam decoding provide additional observed mean reductions. The independent 960-run SPBS study further shows a 26.19% PDI reduction and a rise in incumbent availability from 51.67% to 100%, with both effects significant after Holm adjustment.

The 1500-run stability study further confirms the modeling contribution. Relative to disabling the second stage, the full formulation reduces event-reconstructed route-total and route-maximum wait by 51.91% and 68.89%, and cadence CV and cadence-deviation sum by 39.70% and 94.05%, with all four comparisons remaining significant after Holm adjustment. The factorial study explains where difficulty originates: wafer count and mixed-recipe composition interact strongly, whereas all prespecified processing-scale main and interaction confidence intervals include zero. The 80-run parameter study additionally returns a validated incumbent in every prescribed one-factor perturbation. Together, these results establish the operational advantage of the two-stage formulation and a clear empirical advantage of the structure-aware solver enhancement under the evaluated protocol.

From an operational perspective, the framework separates throughput, schedule refinement, and solver guidance without relaxing the rotating-chamber equipment constraints. Its present scope is deterministic finite-batch operation, and the wall-clock evidence is tied to the evaluated

hardware, instances, and time limits. Extending the model to stochastic processing, online arrivals, module failures, and fab-level rescheduling, and evaluating transfer under larger distribution shifts, are the principal
630 directions for future work.

CRedit authorship contribution statement

The authors jointly contributed to conceptualization, methodology, software, validation, formal analysis, investigation, visualization, and manuscript preparation.

635 Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and code availability

640 The data, mathematical instances, trained models, and source code supporting the findings are available from the corresponding author on reasonable request. A permanent public archive will be cited in the version of record.

645 Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used OpenAI Codex for language editing, consistency checking, and code review. The authors reviewed and edited all AI-assisted output and take full responsibility for the content of the publication.

650 References

Achterberg, T., 2009. SCIP: Solving constraint integer programs. *Mathematical Programming Computation* 1, 1–41. doi:10.1007/s12532-008-0001-1.

- Adelgren, N., Maravelias, C.T., 2023. On the utility of production scheduling formulations including record keeping variables. *Computers & Industrial Engineering* 181, 109330. doi:10.1016/j.cie.2023.109330.
- Bengio, Y., Lodi, A., Prouvost, A., 2021. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research* 290, 405–421. doi:10.1016/j.ejor.2020.07.063.
- Bestuzheva, K., Besançon, M., Chen, W.K., Chmiela, A., Donkiewicz, T., van Doornmalen, J., Eifler, L., Gaul, O., Gamrath, G., Gleixner, A., Gottwald, L., Graczyk, C., Halbig, K., Hoen, A., Hojny, C., van der Hulst, R., Koch, T., Lübbecke, M.E., Maher, S.J., Matter, F., Mühmer, E., Müller, B., Pfetsch, M.E., Schlösser, F., Serrano, F., Shinano, Y., Sofranac, B., Turner, M., Vigerske, S., Wegscheider, F., Wellner, P., Weninger, D., Witzig, J., 2023. Enabling research through the SCIP optimization suite 8.0. *ACM Transactions on Mathematical Software* 49, 22:1–22:21. doi:10.1145/3585516.
- Cappart, Q., Chetelat, D., Khalil, E.B., Lodi, A., Morris, C., Velickovic, P., 2023. Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research* 24, 1–61. URL: <https://www.jmlr.org/papers/v24/21-0449.html>.
- Chen, Z., Liu, J., Wang, X., Yin, W., 2023. On representing mixed-integer linear programs by graph neural networks, in: *Proceedings of the Eleventh International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=4gc3MGZra1d>.
- Chien, C.F., Lan, Y.B., 2021. Agent-based approach integrating deep reinforcement learning and hybrid genetic algorithm for dynamic scheduling for Industry 3.5 smart production. *Computers & Industrial Engineering* 162, 107782. doi:10.1016/j.cie.2021.107782.
- Choo, J., Kwon, Y.D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y., 2022. Simulation-guided beam search for neural combinatorial optimization, in: *Advances in Neural Information Processing Systems*. URL: https://papers.nips.cc/paper_files/paper/2022/hash/39b9b60f0d149eabd1fff2d7c7d5afc4-Abstract-Conference.html, doi:10.52202/068431-0637.

- Gleixner, A., Hendel, G., Gamrath, G., Achterberg, T., Bastubbe, M., Berthold, T., Christophel, P., Jarck, K., Koch, T., Linderoth, J., Lubbecke, M.E., Mittelman, H.D., Ozyurt, D., Ralphs, T.K., Salvagnin, D., Shinano, Y., 2021. MIPLIB 2017: Data-driven compilation of the 6th mixed-integer programming library. *Mathematical Programming Computation* 13, 443–490. doi:10.1007/s12532-020-00194-3.
- Gu, L., Wu, N., Qiao, Y., Zhang, S., 2024. Scheduling of Multi-finger-robotic Cluster Tools with Multi-space Process Modules, in: 2024 International Conference on Networking, Sensing and Control (ICNSC), IEEE. doi:10.1109/ICNSC62968.2024.10760109.
- Hong, T.Y., Hsu, C.Y., Chen, C.L., 2026. Simulation-based deep reinforcement learning for process time optimization in semiconductor cluster tool and an empirical study. *Computers & Industrial Engineering* 211, 111698. doi:10.1016/j.cie.2025.111698.
- Hu, W., Liu, M., Dong, M., Liu, T., Zhang, Y., Cheng, G., 2023. A greedy-based crow search algorithm for semiconductor final testing scheduling problem. *Computers & Industrial Engineering* 183, 109423. doi:10.1016/j.cie.2023.109423.
- Huang, Z., Wang, K., Liu, F., Zhen, H.L., Zhang, W., Yuan, M., Hao, J., Yu, Y., Wang, J., 2022. Learning to select cuts for efficient mixed-integer programming. *Pattern Recognition* 123, 108353. doi:10.1016/j.patcog.2021.108353.
- Khalil, E.B., Morris, C., Lodi, A., 2022. MIP-GNN: A data-driven framework for guiding combinatorial solvers, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 10219–10227. doi:10.1609/aaai.v36i9.21262.
- Kim, D., Kim, H.J., 2025. Noncyclic scheduling of cluster tools using deep reinforcement learning. *IEEE Transactions on Automation Science and Engineering* 22, 24009–24023. doi:10.1109/TASE.2025.3632534.
- Kim, H.J., Lee, J.H., 2025. Scheduling cluster tools for concurrent processing: Deep reinforcement learning with adaptive search. *IEEE Transactions on Automation Science and Engineering* 22, 3783–3796. doi:10.1109/TASE.2024.3399818.

- Kim, W., Yu, T.S., Lee, T.E., 2021. Integrated scheduling of a dual-armed cluster tool for maximizing steady schedule patterns. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 51, 7282–7294. doi:10.1109/TSMC.2020.2978486.
- Ko, S.G., Yu, T.S., Lee, T.E., 2021. Wafer delay analysis and workload balancing of parallel chambers for dual-armed cluster tools with multiple wafer types. *IEEE Transactions on Automation Science and Engineering* 18, 1516–1526. doi:10.1109/TASE.2021.3061140.
- Lee, T.G., Yu, T.S., Lee, T.E., 2023. Cleaning plan optimization for dual-armed cluster tools with general chamber cleaning periods. *IEEE Transactions on Automation Science and Engineering* 20, 1890–1906. doi:10.1109/TASE.2022.3188858.
- Li, C., Chang, Q., 2022. Hybrid feedback and reinforcement learning-based control of machine cycle time for a multi-stage production system. *Journal of Manufacturing Systems* 65, 351–361. doi:10.1016/j.jmsy.2022.09.020.
- Li, C., Yang, F., Zhen, L., 2022. Efficient scheduling approaches to time-constrained single-armed cluster tools with condition-based chamber cleaning operations. *International Journal of Production Research* 60, 3555–3568. doi:10.1080/00207543.2021.1926568.
- Li, X., Yang, W., 2025. Cyclic scheduling of multi-type wafers concurrent processing in single-arm cluster tools with residency time constraints. *Expert Systems with Applications* 269, 126443. doi:10.1016/j.eswa.2025.126443.
- Liu, J., Qiao, F., Zou, M., Zinn, J., Ma, Y., Vogel-Heuser, B., 2022. Dynamic scheduling for semiconductor manufacturing systems with uncertainties using convolutional neural networks and reinforcement learning. *Complex & Intelligent Systems* 8, 4641–4662. doi:10.1007/s40747-022-00844-0.
- Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning, in: *Proceedings of the 33rd International Conference on Machine Learning*, PMLR. pp. 1928–1937. URL: <https://proceedings.mlr.press/v48/mniha16.html>.

- Nemhauser, G.L., Wolsey, L.A., Fisher, M.L., 1978. An analysis of approximations for maximizing submodular set functions—i. Mathematical Programming 14, 265–294. doi:10.1007/BF01588971.
- Pan, C., Lai, Y., Qiao, Y., Wu, N., Luo, X., 2024. Cycle time analysis for cluster tools with parallel process chambers, processing time variation, and chamber cleaning operations. Expert Systems with Applications 255, 124471. doi:10.1016/j.eswa.2024.124471.
- Paulus, M.B., Zarpellon, G., Krause, A., Charlin, L., Maddison, C., 2022. Learning to cut by looking ahead: Cutting plane selection via imitation learning, in: Proceedings of the 39th International Conference on Machine Learning, PMLR. pp. 17584–17600. URL: <https://proceedings.mlr.press/v162/paulus22a.html>.
- Qiao, Y., Lu, Y., Li, J., Zhang, S., Wu, N., Liu, B., 2022. An efficient binary integer programming model for residency time-constrained cluster tools with chamber cleaning requirements. IEEE Transactions on Automation Science and Engineering 19, 1757–1771. doi:10.1109/TASE.2021.3122576.
- Scavuzzo, L., Aardal, K., Lodi, A., Yorke-Smith, N., 2024. Machine learning augmented branch and bound for mixed integer linear programming. Mathematical Programming doi:10.1007/s10107-024-02130-y.
- Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut, in: Proceedings of the 37th International Conference on Machine Learning, PMLR. pp. 9367–9376. URL: <https://proceedings.mlr.press/v119/tang20a.html>.
- Turner, M., Koch, T., Serrano, F., Winkler, M., 2023. Adaptive cut selection in mixed-integer linear programming. Open Journal of Mathematical Optimization 4, 1–28. doi:10.5802/ojmo.25.
- Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks, in: Advances in Neural Information Processing Systems, pp. 2692–2700.
- Wang, J., Liu, C., Zhou, M., Leng, T., Albeshri, A., 2023a. Optimal cyclic scheduling of wafer-residency-time-constrained dual-arm cluster tools by

- 785 configuring processing modules and robot waiting time. *IEEE Transactions on Semiconductor Manufacturing* 36, 251–259. doi:10.1109/TSM.2023.3239198.
- Wang, J., Wang, Z., Li, X., Kuang, Y., Shi, Z., Zhu, F., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2024. Learning to cut via hierarchical sequence/set model for efficient mixed-integer programming. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 9697–9713. doi:10.1109/TPAMI.2024.3432716.
- 790 Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y., Wu, F., 2023b. Learning cut selection for mixed-integer linear programming via hierarchical sequence model, in: *Proceedings of the Eleventh International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Zob4P9bRNcK>.
- Yang, F., Li, C., Zhen, L., Zhang, C., Wu, N., 2025. Scheduling residency time-constrained single-armed cluster tools with two-wafer types via mixed integer programming model. *Expert Systems with Applications* 288, 128209. doi:10.1016/j.eswa.2025.128209.
- 800 Yuan, F., Zhao, Q., Huang, B., Pan, C., 2023. Scheduling of time-constrained single-arm cluster tools with purge operations in wafer fabrications. *Journal of Systems Architecture* 134, 102788. doi:10.1016/j.sysarc.2022.102788.
- 805 Zhang, L., Lin, Y., Xu, C., Liu, M., 2024. A new EDA algorithm combined with Q-learning for semiconductor final testing scheduling problem. *Computers & Industrial Engineering* 193, 110259. doi:10.1016/j.cie.2024.110259.
- Zhu, Q., Li, B., Hou, Y., Li, H., Wu, N., 2023. Scheduling dual-arm multi-cluster tools with regulation of post-processing time. *IEEE/CAA Journal of Automatica Sinica* 10, 1730–1742. doi:10.1109/JAS.2023.123189.
- 810 Zhu, Q., Wang, G., Hou, Y., Wu, N., Qiao, Y., 2022. Optimally scheduling dual-arm multi-cluster tools to process two wafer types. *IEEE Robotics and Automation Letters* 7, 5920–5927. doi:10.1109/LRA.2022.3157031.

- 815 Zhu, Q., Zhang, G., Hou, Y., Wu, N., 2026. Scheduling a dual-arm robotic two-cluster tool in the event of a process module failure. *Expert Systems with Applications* 297, 129415. doi:10.1016/j.eswa.2025.129415.
- Zou, C., Zhang, S., Zeng, S., Gu, L., Li, J., 2024. Wafer delay minimization in scheduling single-arm cluster tools with two-space process modules. 820 *Mathematics* 12, 1783. doi:10.3390/math12121783.